

Web Design Group Members:

HIRWA Elysee **29058**

RWABUNEZA Mark **28750**

NKUBANA Guy **29056**

INEMA SENTAMA Laure **28954**

UWASE Cynthia **29583**

NIYOKWIZIGIRWA Carmel

System Documentation: User Profile Management System

1. Executive Summary & Overview

The **User Profile Management System** is a responsive, database-driven web application developed in PHP and MySQL. It provides complete account lifecycle functionality—including user registration, authentication, session-based profile access, account updates, and account deletion—while maintaining a semantic HTML5 structure styled through CSS.

The system uses server-side processing to handle business logic, database transactions, form validation, and session management.

Each file of the system has a .php extension to allow integration of PHP in the HTML code. This integrated PHP code allows to check session information before the page loads.

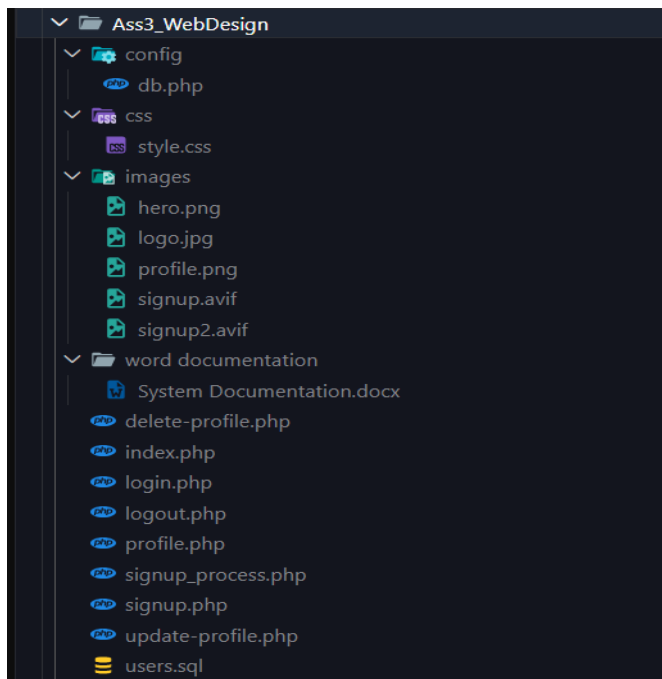
2. Database Architecture (users.sql)

The database consists of a single relational table named **users** stored inside the **user_profile_system** database schema.

Table Structure: users

Field Name	Data type	Constraints	Description
ID	INT(11)	PRIMARY KEY, AUTO_INCREMENT, NOT NULL	Unique numeric identifier for each user.
full_name	VARCHAR(100)	NOT NULL	User's complete name.
email	VARCHAR(100)	UNIQUE KEY, NOT NULL	User's unique email address (used for login
username	VARCHAR(50)	UNIQUE KEY, NOT NULL	Unique user handler.
password	VARCHAR(255)	NOT NULL	Securely hashed password string
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Account creation timestamp.
updated_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP	Automatic update timestamp on record modification.

3. System Architecture & File Structure



4. Technical Component Breakdown

4.1. Database Connection Configuration (config/db.php)

- **Purpose:** Establishes a connection between the PHP runtime engine and the MySQL database server. It was put outside, to be re-used multiple times without re-writing of the code.
- **Execution Flow:**
 1. Defines connection constants: `$servername = "localhost"`, `$username = "root"`, `$password = ""`, `$dbname = "user_profile_system"`.
 2. Initializes a connection instance using `mysqli_connect()`.
 3. Checks `$conn->connect_error`; if active, stops the script execution via `die()`.
 4. Sets character encoding to UTF-8 (`mysqli_set_charset($conn, "utf8")`) to prevent character encoding mismatch issues.

4.2. Home Page (index.php)

- **Purpose:** Landing page listing features, user guidance, and entry points. It also had the Call to Action of the website.
- **Layout Structure:** Built using semantic HTML5 tags including `<header>`, `<nav>`, `<main>`, `<article>`, `<section>`, `<aside>`, and `<footer>`.
- **Key Functionality:** Starts session tracking (`session_start()`) and displays navigation links reflecting account interaction entry points.

4.3. User Registration (signup.php & signup_process.php)

- **Purpose:** Collects new user records, executes multi-stage validation, hashes passwords, and writes data to MySQL database once validated.
- **Execution Flow:**
 1. **UI (signup.php):** Renders a split-container with one side as an image and another as a form taking `full_name`, `email`, `username`, `password`, and `confirm_password` inputs.
 2. **Processing (signup_process.php):** Accepts input via HTTP POST request and is responsible for the database manipulation and query handling.

3. Validation:

- Verifies all inputs are non-empty.
 - Validates email syntax using `filter_var($email, FILTER_VALIDATE_EMAIL)`.
 - Verifies if the password and confirm password are the same.
 - Enforces minimum password length constraint (`strlen($password) >= 6`).
 - Checks database for existing duplicate email or username entries.
4. **Persistence & Encryption:** Hashes password using `password_hash($password, PASSWORD_BCRYPT)`. Executes an INSERT INTO users query.
5. **Session Initiation:** On successful insertion, assigns `$_SESSION` credentials (`user_id`, `username`, `email`, `full_name`) and redirects to `profile.php`.

4.4. User Authentication (login.php)

- **Purpose:** Authenticates existing users by cross referencing the data in the database and initiates secure user sessions once validated to be true.
- **Execution Flow:**
 1. Renders login form collecting email and password.
 2. Validates non-empty input fields.
 3. Queries database for existing user record matching supplied email.
 4. Verifies supplied plain-text password against stored hash using `password_verify($password, $hashed_password)`.
 5. On successful match, populates `$_SESSION` variables and redirects to `profile.php`.

4.5. User Profile View (profile.php)

- **Purpose:** Displays authenticated user details retrieved directly from the database.
- **Execution Flow:**
 1. **Authentication Check:** Verifies if the session id is populated; redirects unauthenticated visitors to `login.php`.
 2. **Database Fetch:** Runs `SELECT full_name, email, username, created_at FROM users WHERE id = '$user_id'`.

3. **UI Display:** Formats user details inside a structured <article> tag. Provides action routes to update or delete account.

4.6. Profile Update (update-profile.php)

- **Purpose:** Allows logged-in users to modify full_name, email, or username records.
- **Execution Flow:**
 1. **GET Processing:** Loads current profile data into form input value attributes.
 2. **POST Processing:** Validates submitted data for empty values and proper email format.
 3. **Collision Avoidance:** Ensures new email or username are not already registered to *other* accounts using WHERE email = '\$email' AND id != '\$user_id'.
 4. **Database Modification:** Executes UPDATE users SET full_name = ..., email = ..., username = ... WHERE id = '\$user_id'.
 5. **Session Refresh:** Updates active \$_SESSION variables and redirects to profile.php with a success message.

4.7. Profile Deletion (delete-profile.php)

- **Purpose:** Removes user records from the system.
- **Execution Flow:**
 1. Validates logged-in user session state.
 2. Executes DELETE FROM users WHERE id = '\$user_id'.
 3. Destroys active session (session_destroy()).
 4. Re-initializes a temporary session to output an account deletion message on index.php.

4.8. Session Termination (logout.php)

- **Purpose:** Terminates user authentication state.
- **Execution Flow:** Runs session_start(), executes session_destroy() to wipe stored session values, and redirects user to index.php.

5. Security & Design Patterns Applied

1. **Password Security:** Uses **BCRYPT** hashing algorithm (PASSWORD_BCRYPT) for storage, preventing plain-text credential leaks.
2. **Access Control:** Every protected script (profile.php, update-profile.php, delete-profile.php) verifies active session tokens (\$_SESSION['user_id']) before processing requests or rendering protected data hence making all files .php to allow PHP code into HTML structure.
3. **Session Message Flashing:** Implements single-use error/success messages stored in \$_SESSION['error'] or \$_SESSION['success'], which are automatically cleared via unset() after rendering.
4. **Responsive CSS Design:** Implements Flexbox layouts (display: flex), media queries (@media (max-width: 768px)), and modern form styling rules for seamless desktop and mobile accessibility.